

module 'Votes'

Actions:

1. [createVote](#)
2. [createMixVote](#)
3. [getTriviaQuestions](#)

createVote

create a widget for voting.

key: 'createVote'

payload:

required fields:

title: string;

type: string; ('binary', 'target_point', 'target_poll')

question: string;

terminationCondition: string; (voting will end when:

'time', - **duration** runs out

'get_all_votes', - all actors have voted

'get_needed_votes', - one of the voting options will score

neededVotes

'get_majority', - one of the voting options will score 50% + 1 of actors

number

'get_two_thirds' - one of the voting options will score $\frac{2}{3}$ of actors

number)

actors: Array of strings; (user or teams ids)

showResultInRealTime: boolean; (if 'true' the widget displays current vote results)

optional fields:

scope: 'team'; (for using teams as actors)

duration: number; (in seconds, if the value is set voting will be interrupted after the duration expires. Needed when terminationCondition='time')

targets: Array of strings; (Needed when type='target_point' or 'target_poll'. User ids for 'target_point'.)

private: boolean; (if 'true' vote widget is visible for actors only)

showResult: boolean; (if 'true' the voting result is shown after the end of voting)

showResultDelay: number; (in seconds, delay between the end of voting and appearance of the voting result)

showResultDuration: number; (in seconds, the voting result will be displayed for the set number of seconds. Needed when **showResult**=true)

allowSkipping: boolean; (if 'true' allows the player to skip the vote)

allowRevoting: boolean; (if 'true' allows the player to change their vote)

neededVotes: number; (needed when terminationCondition='get_needed_votes')

neededAnswer: string; (certain answer for **neededVotes**)

answersQuantity: number; (For type='target_point' or 'target_poll' determines the number of votes a player has to send. Used for multi choice voting. default 1.)

allowFewerAnswers: boolean; (if 'true' a player can send fewer votes than answersQuantity)
revealResults: boolean; (if 'false' the result is not shown in the game history. default 'true')
suppressHistory: boolean; (if 'true' the history item for that action is not shown, default 'false');

binaryAnswerAliases: [string, string]; (if type='binary' allows to change the names of positive and negative answers (by default ['yes', 'no']))

oneClick: boolean; (If 'true', then instead of needing to confirm a vote or unvote on another player (which is 2 clicks - 1 click to click on the box of a player, another click to confirm the vote/unvote), you only need 1 click to do this. 'false' by default)

pollVoteTargetsOptions: record like {

```
target: {
    textColor: string; (as CSS value)
    backgroundColor: string; (as CSS value)
    widgetIconColor: string; (as CSS value)
    boxIconColor: string; (as CSS value)
    icon: string; (URI)
},
```

...

```
}
```

allows you to customize the display of widget buttons and icons.

default for 'target_poll' {

```
    textColor: '#7b7b7b',
    backgroundColor: 'white',
    widgetIconColor: 'black',
    boxIconColor: 'black',
}
```

default for 'binary' {

yes: {

```
    textColor: 'white',
    backgroundColor: '#55AA64',
    widgetIconColor: '#7B7B7B',
    boxIconColor: '#55AA64',
    icon:
```

```
'https://res.cloudinary.com/liars-club/image/upload/v1657512113/icons/like.svg',
},
```

no: {

```
    textColor: 'white',
    backgroundColor: '#DA5050',
    widgetIconColor: '#7B7B7B',
    boxIconColor: '#DA5050',
    icon:
```

```
'https://res.cloudinary.com/liars-club/image/upload/v1657512182/icons/dislike.svg',
```

```
},
```

```
}
```

hideOptionsFromNonActors: boolean; (If the field isn't present or 'false' everyone sees the targets for public votes.)

backgroundColor: string; (CSS value for widget)

textColor: string; (CSS value for widget)

borderColor: string; (CSS value for widget)

vertical: boolean;

image: string;

returns result object:

```
{
  voteResult: array of strings (answers); - Answers with the most votes
  voices: {[userId]: array of strings (answers)}; - Record of answers by player
  answersCount: {[answer]: number}; - Record of number of votes by answer
  answersVoters: {[answer]: array of strings (userId)}; -Record of users who voted for
  answer
}
```

createMixVote

create a widget for mix voting.

key: 'createMixVote'

payload:

required fields:

title: string;

question: string;

terminationCondition: string; (voting will end when:

'time', - **duration** runs out

'get_all_votes', - all actors have voted

'get_needed_votes', - one of the voting options will score

neededVotes)

actors: Array of strings; (user or teams ids)

showResultInRealTime: boolean; (if 'true' the widget displays current vote results)

'point.targets': Array of strings; (user ids)

'poll.targets': Array of strings;

optional fields:

scope: 'teams'; (for using teams as actors)

duration: number; (in seconds, if the value is set voting will be interrupted after the duration expires. Needed when terminationCondition='time')

'point.answersQuantity': number; (determines the number of votes a player has to send for point targets. Default 1.)

'point.allowFewerAnswers' boolean; (if 'true' a player can send fewer votes than point.answersQuantity)

'point.terminationCondition: string; (voting will end when:

'get_majority', - one of the voting options will score 50% + 1 of actors number

'get_needed_votes', - one of the voting options will score

neededVotes)

'point.neededVotes': number; (needed when 'point.terminationCondition'='get_needed_votes')

'poll.answersQuantity': number; (determines the number of votes a player has to send for poll targets. Default 1.)

'poll.allowFewerAnswers': boolean; (if 'true' a player can send fewer votes than poll.answersQuantity)

'poll.terminationCondition': string; (voting will end when:
 'get_majority', - one of the voting options will score 50% + 1 of actors number
 'get_needed_votes', - one of the voting options will score neededVotes)

'poll.neededVotes': number; (needed when 'poll.terminationCondition'='get_needed_votes')

private: boolean; (if 'true' vote widget is visible for actors only)

showResult: boolean; (if 'true' the voting result is shown after the end of voting)

showResultDelay: number; (in seconds, delay between the end of voting and appearance of the voting result)

showResultDuration: number; (in seconds, the voting result display duration. Needed when showResult=true)

allowSkipping: boolean; (if 'true' allows the player to skip the vote)

allowRevoting: boolean; (if 'true' allows the player to change the decision)

neededVotes: number; (needed when terminationCondition='get_needed_votes')

allowFewerAnswers: boolean; (if 'true' a player can send fewer votes than answersQuantity)

revealResults: boolean; (if 'false' the result is not shown in the game history. default 'true')

suppressHistory: boolean; (if 'true' the history item for that action is not shown, default 'false');

oneClick: boolean; (If 'true', then instead of needing to confirm a vote or unvote on another player (which is 2 clicks - 1 click to click on the box of a player, another click to confirm the vote/unvote), you only need 1 click to do this. 'false' by default)

canVoteOnBothOptions: boolean; (If true, you can vote for the poll and point options at the same time)

pollVoteTargetsOptions: record like {
 target: {
 textColor: string; (as CSS value)
 backgroundColor: string; (as CSS value)
 widgetIconColor: string; (as CSS value)
 boxIconColor: string; (as CSS value)
 icon: string; (URI)
 },
 ...
}
(allows you to customize the display of widget buttons and icons.
default {
 textColor: '#7b7b7b',
 backgroundColor: 'white',
 widgetIconColor: 'black',
 boxIconColor: 'black',
}

hideOptionsFromNonActors: boolean; (If the field isn't present or 'false' everyone sees the targets for public votes.)

backgroundColor: string; (CSS value for widget)

textColor: string; (CSS value for widget)

borderColor: string; (CSS value for widget)

vertical: boolean;

image: string;

returns result object:

```
{
  voteResult: array of strings (answers); - Answers with the most votes
  voices: {[userId]: array of strings (answers)}; - Record of answers by player
  answersCount: {[answer]: number}; - Record of number of votes by answer
  answersVoters: {[answer]: array of strings (userId)}; -Record of users who voted for
  answer
}
```

getTriviaQuestions

key: 'getTriviaQuestions'

payload:

required fields:

qnt: number; (how many questions we need)

optional fields:

category: string;

minDifficulty: number; (the minimum acceptable difficulty between 1 and 5)

maxDifficulty: number; (the maximum acceptable difficulty between 1 and 5)

The action result is array of records:

```
{
  question: string;
  options: string[];
  answer: number;
}
```